

Dashboard "Pague-se Primeiro"

Documento de Especificação de Arquitetura e Engenharia de Software

Autor: Engenheiro de Software FullStack & Jarvis | Stack: HTML5, CSS3, JavaScript Vanilla (ES6+)

1. Visão Geral do Produto

O **Dashboard "Pague-se Primeiro"** é um micro-SaaS focado em finanças comportamentais. Diferente dos gerenciadores financeiros tradicionais, que geram atrito e sensação de escassez ao exigir que o usuário catalogue cada pequeno gasto, este software inverte o paradigma comportamental. Ele baseia-se na premissa de acumulação de ativos de clássicos da literatura financeira, estabelecendo como métrica de sucesso o ato de "pagar a si mesmo primeiro" antes de efetuar o pagamento de qualquer passivo ou conta.

Objetivo Estratégico para o Portfólio: Demonstrar proficiência avançada em JavaScript Vanilla sem dependência de frameworks modernos (como React, Vue ou Angular). O projeto evidencia o domínio de conceitos fundamentais da linguagem, manipulação performática do Document Object Model (DOM), gerenciamento de estado previsível e persistência de dados do lado do cliente.

2. Conceito de Negócio e Psicologia Financeira

A aplicação utiliza gatilhos mentais e mecanismos de gamificação para criar hábitos financeiros saudáveis:

- **Gatilho de Relação Dor vs. Prazer:** O foco muda da dor de gastar para o prazer visual de ver o patrimônio líquido crescer de forma cumulativa.
- **Reforço Positivo Dinâmico:** Uso de feedback visual instantâneo baseado em metas. O sistema calcula automaticamente se o aporte mensal atingiu a meta mínima de 10% da renda total. Se a meta for superada, a interface reage com tons dourados/verdes (sucesso); caso contrário, adota tons neutros ou de alerta (cinza/vermelho).

3. Arquitetura de Software e Fluxo de Dados

A aplicação adota um padrão de **Fluxo de Dados Unidirecional** adaptado para o ecossistema Vanilla JS. O estado da aplicação atua como a única fonte da verdade (*Single Source of Truth*), garantindo previsibilidade e facilidade de depuração.

O ciclo do fluxo de dados opera da seguinte forma:

1. O usuário interage com a interface (UI), disparando um evento nativo do navegador.

2. O Orquestrador captura o evento e invoca uma função de mutação no Gerenciador de Estado.
3. O Estado é atualizado seguindo regras de negócio rígidas e aciona automaticamente o módulo de Armazenamento para persistência.
4. A Interface é notificada (ou atualizada pelo orquestrador) para renderizar novamente os componentes visuais com base no novo Estado.

4. Estrutura de Diretórios (Clean Architecture)

A organização dos arquivos segue princípios de design limpo, segregando responsabilidades em módulos isolados via ES6 Modules (`import/export`):

```
pagar-se-primeiro/  
├─ index.html      # Estrutura semântica, acessibilidade e carregamento de  
CDNs  
├─ css/  
│   └─ styles.css  # Design System global, variáveis (CSS Custom Properties)  
e resets  
│   └─ dashboard.css # Estilos específicos de componentes (cards, painéis,  
inputs)  
└─ js/  
    └─ app.js       # Ponto de entrada (Entrypoint) e orquestrador de eventos  
    └─ state.js      # Core de negócios: gerenciamento de estado e regras  
financeiras  
    └─ storage.js    # Camada de abstração e isolamento do LocalStorage  
    └─ ui.js         # Manipulação do DOM e renderização gráfica (Chart.js)
```

5. Detalhamento Técnico dos Módulos

Módulo	Responsabilidade Técnica	Principais Funções / Métodos
storage.js	Abstrair a API de armazenamento do navegador. Isola o código de negócio do mecanismo físico de persistência, permitindo migração futura para APIs de nuvem ou bancos NoSQL sem alterar o restante do app.	<code>saveState(state)</code> <code>loadState()</code>
state.js	Manter o estado mutável em memória e aplicar as validações de regras de negócio. Calcula percentuais de aporte, valida tipos de dados de entrada e computa as métricas globais de performance financeira.	<code>adicionarMes(renda, economizado)</code> <code>calcularMetricas()</code>
ui.js	Interagir diretamente com a árvore do DOM. Responsável pela atualização cirúrgica de textos, mutação de classes CSS e inicialização/destruição de instâncias de gráficos através da biblioteca Chart.js.	<code>renderDashboard(state)</code> <code>renderChart(registros)</code>
app.js	Atuar como a cola arquitetural. Inicializa o aplicativo carregando os dados do storage, injeta o estado inicial e vincula os Event Listeners (como submits de formulários) às ações de mutação de estado.	<code>init()</code> <code>handleFormSubmit(e)</code>

6. Modelagem de Dados (The State Object)

O objeto que rege o estado da aplicação possui uma estrutura estritamente tipada de forma conceitual, facilitando o entendimento de como as informações transitam:

```
const appState = {
  metaEconomia: 10, // Porcentagem padrão recomendada (10%)
  registrosMensais: [
    {
      id: "1685536800000", // Timestamp único para identificação
      mes: "Janeiro",
    }
  ]
}
```

```
renda: 5000.00,  
economizado: 600.00 // Aporte de 12% (Sucesso)  
},  
{  
  id: "1688128800000",  
  mes: "Fevereiro",  
  renda: 6000.00,  
  economizado: 400.00 // Aporte de 6.6% (Abaixo da Meta)  
}  
]  
};
```

7. Estratégia de Posicionamento e Venda de Portfólio

Para maximizar o impacto deste projeto frente a recrutadores e líderes técnicos, utilize as seguintes diretrizes em seu repositório (README.md) e entrevistas:

1. **Separação de Conceitos (SoC):** Enfatize que o projeto adota os mesmos princípios arquiteturais usados por grandes corporações em aplicações React/Redux, provando que você compreende a engenharia por trás das ferramentas antes de simplesmente utilizá-las.
2. **Ausência de Frameworks como Decisão Arquitetural:** Defenda que a escolha de Vanilla JS foi estratégica para demonstrar proficiência com escopo de variáveis, closures, manipulação avançada de arrays (`.map` , `.reduce` , `.filter`) e consumo assíncrono de módulos nativos.
3. **Foco em Performance e Custo Zero:** Explique que a aplicação roda inteiramente no lado do cliente (Client-Side), garantindo carregamento instantâneo, segurança absoluta da informação financeira do usuário (já que nenhum dado sensível sai do seu próprio navegador) e custo zero de infraestrutura e servidores.